

AIR UNIVERSITY
Department of Electrical
College of Engineering

ARTIFICIAL INTELLIGENCE LAB

LAB 3

Lab Objective: Local Search Algorithms - Gradient Descent Optimization

Class ID: _____

Lab Date: _____

Student's ID: _____

LAB 3	Local Search Algorithms - Gradient Descent Optimization				
PLO's	PLO3 – Design and Development		Bloom's Taxonomy	C3 – Apply	
LAB TASK PERFORMANCE					
CLO's	Aspects of Assessments	Excellent (75-100%)	Average (50-75%)	Poor ($<50\%$)	M
CLO2 100%	<u>Design & Development</u> Design solution for problem that meets specified needs by using the concept of artificial intelligence.	A complete solution / Explain necessary requirements according to task description with great use of time and resource material.	Solution was complete but need minor modifications /student could have followed specification more closely.	Solution was complete but did not work, needed several modifications / did not make correct use of resource material or instructions.	
Total Marks: 10					

Graded by Engineer: _____

Date: _____

Remarks: _____

Objective:

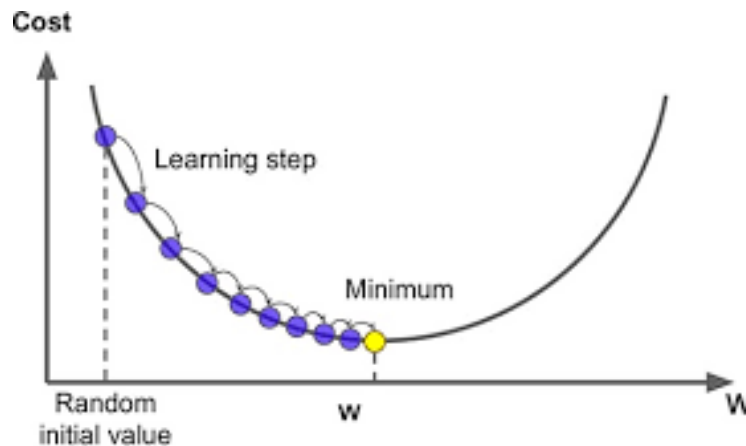
1. Make attendee familiar with the concept of local search algorithms

2. Elaborate the fundamental principle of gradient descent optimization (also called as steepest descent)
3. Hands-on implementation on Python using the gradient descent optimization

Gradient Descent Optimization

Gradient descent is a **first-order iterative optimization algorithm** for finding a **local minimum** of a differentiable function. To find a local minimum of a function using gradient descent, we take steps proportional to the *negative* of the **gradient** (or approximate gradient) of the function at the current point. But if we instead take steps proportional to the *positive* of the gradient, we approach a **local maximum** of that function; the procedure is then known as gradient ascent.

We need to find minima of a simple bowl-shaped function, $L(w)$, as shown below.



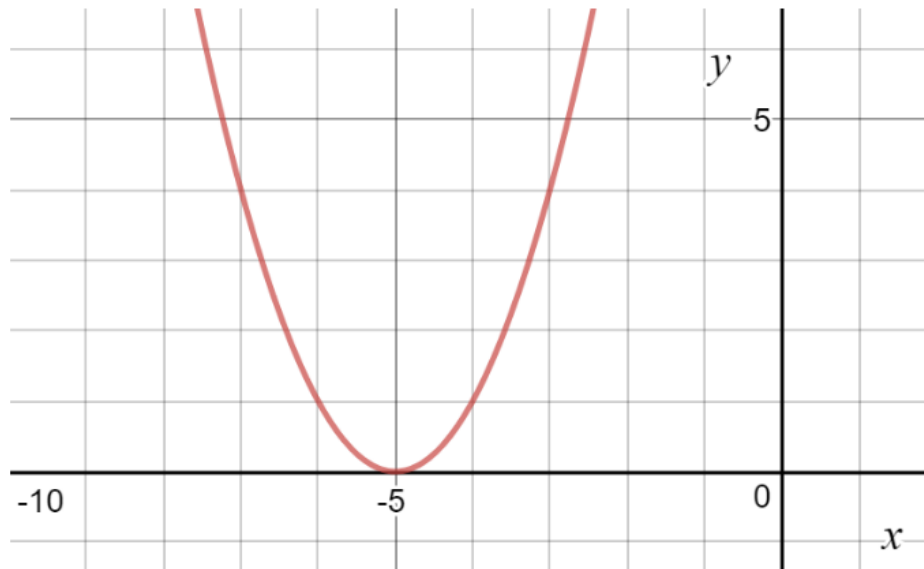
1. We start randomly at any point and keep changing the values of 'w' to move down-hill in small steps until we reach the minimum
2. Note that the parameter 'w' could have any notation like x, θ etc
3. The size of steps we take is called as "learning rate" or "step-size". It is typically denoted by Latin alphabets α or η
4. Gradient at any point of the curve gives you positive slope
5. Since we need to go down-hill we have to move in the negative direction of the gradient
6. Update rule of GD:
7. (New position):= (old position) – (step-size) * (gradient of the function)
8. This is called as the first iteration of the GD algorithm
9. We will continue to do this until hopefully we reach the minima of our function
10. Mathematically we can write:

$$w_{new} = w - \alpha * \frac{\delta L}{\delta w}$$

11. This is called as the update rule of gradient descent

Hands-on Activity

In this exercise you will learn how to implement a gradient descent (GD) algorithm. It is really imperative to understand the working of the GD algorithm as many data science/ML algorithms are based on this concept. You will try to find the minimum of a concave function $y=(x+5)^2$ starting with an initial value $x=2$ for example. The function looks like this:



```
#initialize parameters ie x and eta (two lines of code)
x=2
eta=0.01 ## We have assumed a typical value for the learning rate
for i in range(1):
    x=x-eta*(2*(x+5))
    print(x)
```

1.8599999999999999

Now let's increase the iterations to say 500 and see how things go..

```
[ ]
#initialize parameters ie x and eta (two lines of code)
x=2
eta=0.01 ## We have assume a typical value for the learning rate
for i in range(500):
    x=x-eta*(2*(x+5))
    print(x)
```

Expected value: -4.999712832103982

Lab Task:

(CLO2, Marks 4) Complete hands-on activities as prescribed above. What effect has learning rate (Step size) on the convergence of algorithm. Justify your answer with mathematical reasoning

Now, for the following higher order polynomial functions repeat above experiments varying learning rate (0.0001, 0.001, 0.01, 0.1) and number of iterations (1, 2, 5, 50, 100, 500). (Note keep one parameter fixed and vary the other and then repeat).

1. $y = 3(x-5)^4 + 8(x-5)^3 - 18(x-5)^2$
2. $y = 6x^3 - 12x^2 + 1$
3. $y = 4x/(1+x^2)$
4. $y = 4x^{0.5} - x^2$

Comment on the behavior of the algorithm convergence as you vary the learning rate and iterations. Plot a table for each polynomial function as per above variation in learning rate and number of iterations.

How does the algorithm behave at points of inflection? Is the algorithm always successful in finding the global minima or sometimes find the local minima only. (**Note only positive values of x are to be used with $x_{\max}$ fixed to 300**). Comment on the answers obtained with proper justification.

(CLO 6, Marks 1) Perform properly handling of lab.

Post Lab Task:

In this task you will get a chance to perform feature gradient descent minimization of a two dimensional cost function:

$$J(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2$$

1. (CLO2, Marks 2) Find the update rule for θ_1 and θ_2 .
2. (CLO2, Marks 3) Now minimize the above cost function using the concept of gradient descent minimization. You may initialize your code with $\theta_1 = \theta_2 = 0.75$, take learning rate = 0.2. Your code should converge in 10 iterations as such. Report the final values of the parameters θ_1 and θ_2
3. Comment on the answer
4. You vary the initialization values, learning rate and number of iterations. What difference do you see in the rate of convergence of algorithm.